SECURITY TECHNICAL REFERENCE • ARCHITECTURE SPECIFICATION

UNSW-NB15 Dataset Technical Reference & Feature Specification

A comprehensive technical manual detailing all 36 baseline network flow metrics, physical transport semantics, attack attribution signatures, and the 48 engineered domain features powering the X-NIDS Explainable AI detection engine.

PROJECT CORE	DATASET RECORDS	FEATURE SPACE	DETECTION PARADIGM
X-NIDS Engine	82,332 NetFlows	36 Raw • 48 Engineered	Ensemble + SHAP XAI

SEC 1 Dataset Architecture & Attack Distribution Profile

The **UNSW-NB15** dataset was created by the Australian Centre for Cyber Security (ACCS) using the Cyber Range Lab of UNSW Canberra. Raw bidirectional packet traces were captured using the IXIA PerfectStorm tool, synthesizing realistic modern background traffic with comprehensive attack profiles. Traces were processed via **Bro-IDS (Zeek)** and **Argus** flow analyzers, producing 36 rich statistical features spanning connection dynamics, timing, throughput, and protocol flags.

> **Data Sanitization Protocol in X-NIDS:** Prior to model training, our pipeline identified and purged **1,914 feature-identical conflicting records** (instances with identical 34 network features but discordant ground-truth labels) and deduplicated **32,361 identical records** to prevent decision tree memorization and artificial metric inflation.

Complete Attack Topology & Distribution in Dataset (82,332 Cleaned Flow Records)

Class Category	Instances	Share	Severity Profile	Physical Attack Mechanism & Observable Network Behavior
Normal	37,000	44.94%	BENIGN Standard	Legitimate user transactions: HTTP/HTTPS web browsing, DNS lookups, SMTP mail, and authorized file transfers. Characterized by balanced handshake latencies, complete TCP teardowns (`FIN`), and normal payload ratios.
Generic	18,871	22.92%	HIGH Cryptographic	Collision technique targeting block ciphers and hash functions without internal structure knowledge. Generates high packet velocity with uniform, predictable payload byte lengths.
Exploits	11,132	13.52%	CRITICAL Vulnerability	Exploits known CVE vulnerabilities in daemons and operating systems to alter execution flow. Manifests in abnormal URI request strings, irregular packet sizes, and unusual state sequences.
Fuzzers	6,062	7.36%	MEDIUM Automated	Automated testing injecting pseudo-random, boundary-breaking inputs to crash servers. Characterized by elevated connection resets (`RST`), high jitter, and anomalous mean packet lengths.
DoS	4,089	4.97%	HIGH Exhaustion	Denial of Service attacks designed to saturate network bandwidth or deplete OS socket buffers. Exhibits extreme asymmetry: high `sbytes` / `spkts`, zero or negligible `dbytes`, and astronomical packet rates.
Reconnaissance	3,496	4.25%	MEDIUM Probing	Port sweeps and service footprinting (Nmap, masscan). Identified by ultra-short durations (`dur ~ 0`), elevated `ct_dst_sport_ltm`, and interrupted/unreplied connection states (`REQ` / `INT`).
Analysis	677	0.82%	MINORITY Web Probing	Specialized web application probes (SQL injection, XSS parameter fuzzing, path traversal). Characterized by custom HTTP query verbs and short pipelined transaction depths.
Backdoor	583	0.71%	MINORITY Covert C2	Covert persistence mechanism bypassing standard authentication for command-and-control (C2). Exhibits periodic beaconing inter-packet timing, low byte volume, and extended flow durations.
Shellcode	378	0.46%	MINORITY Payload Exec	Machine-code byte sequences transmitted to exploit memory flaws and spawn command shells. Characterized by specific byte lengths, small packet counts, and distinct TCP sequence offsets.
Worms	44	0.05%	ULTRA-RARE Propagation	Self-replicating malicious programs propagating autonomously across network subnets by scanning and infecting vulnerable peer hosts. Highly bursty across sequential destination ports.

SEC 2 Comprehensive Specification of the 36 Baseline Dataset Features

The baseline features capture 5-tuple connection semantics, transport-layer statistics, flow dynamics, jitter variance, and application-layer flags. The first 15 features are detailed below:

1. Connection Identification & Protocol Dynamics (4 Features)

Feature	Type	Metric Definition & Units	Physical Network Semantics & Security Analysis
`dur`	`float64`	**Flow Duration:** Total connection elapsed time in seconds ($t_{end} - t_{start}$).	Distinguishes rapid port scans ($dur < 0.001s$) from sustained connections. Persistent backdoors and data exfiltration exhibit abnormally extended duration values.
`proto`	`string`	**Transaction Protocol:** Transport/network layer protocol (`tcp`, `udp`, `icmp`, `arp`, etc.).	Identifies protocol-specific anomaly signatures (e.g., UDP amplification floods vs. TCP SYN floods). 133 unique protocols appear across UNSW-NB15 traces.
`service`	`string`	**Application Service:** Identified protocol service (`http`, `ftp`, `smtp`, `ssh`, `dns`, or `"-"`).	Permits context-aware inspection. Attacks targeting non-standard services or disguising malicious payloads on unassigned ports appear as `"-"`.
`state`	`string`	**Connection State:** Protocol state indicator (`FIN`, `CON`, `INT`, `REQ`, `RST`, etc.).	Normal TCP concludes with `FIN`. Floods and SYN scans produce `REQ` (unreplied) or `INT` (interrupted before 3-way handshake completes).

2. Packet & Byte Volume Metrics (6 Features)

Feature	Type	Metric Definition & Units	Physical Network Semantics & Security Analysis
`spkts`	`int64`	**Source Packet Count:** Total packets sent from client initiator to destination host.	High `spkts` combined with low responder packets indicates unidirectional DoS attacks or brute-force credential stuffing attempts.
`dpkts`	`int64`	**Destination Packet Count:** Total packets returned from responder to source.	Zero `dpkts` indicates rejected, dropped, or firewalled connections—a primary fingerprint of reconnaissance port scans and SYN floods.
`sbytes`	`int64`	**Source Bytes:** Total byte volume transmitted by source host including headers and payload.	Captures upload volume. Unusually large source byte volumes point toward data exfiltration, memory corrupting buffer overflows, or heavy file uploads.
`dbytes`	`int64`	**Destination Bytes:** Total byte volume transmitted by destination host.	Captures server response size. Disproportionately massive `dbytes` relative to request size flags data leakage or large exploit payloads retrieved.
`smean`	`int64`	**Source Mean Packet Size:** Average size of packets sent from source ($sbytes / spkts$).	Probing tools use minimal frame headers ($smean \approx 40\text{--}64$ bytes). Malware with embedded payloads creates large, uniform packet means.
`dmean`	`int64`	**Destination Mean Packet Size:** Average size of packets sent from destination ($dbytes / dpkts$).	Differentiates small ACK responses ($dmean \approx 40$ bytes) from rich content transfers (HTML, binary payloads) in server replies.

3. Rate, Throughput & Packet Loss Dynamics (5 Features)

Feature	Type	Metric Definition & Units	Physical Network Semantics & Security Analysis
`rate`	`float64`	**Transmission Rate:** Aggregate packets per second: $(spkts + dpkts) / dur$.	Critical velocity indicator. Extreme rates characterize automated DoS floods, while stealthy persistent attacks deliberately suppress this metric.
`sload`	`float64`	**Source Bit Rate:** Source throughput in bits/second: $(sbytes \times 8) / dur$.	Bandwidth saturation metric. Volumetric attacks generate astronomical source bit rates designed to exhaust uplink capacity.
`dload`	`float64`	**Destination Bit Rate:** Destination throughput in bits/second: $(dbytes \times 8) / dur$.	Evaluates return bandwidth. Amplification attacks (e.g., DNS/NTP reflection) trigger massive asymmetric destination load values.
`sloss`	`int64`	**Source Packet Loss:** Dropped or retransmitted packets by initiator.	Signals transmission degradation, aggressive congestion avoidance, or network stack interference by network defenses.
`dloss`	`int64`	**Destination Packet Loss:** Dropped or retransmitted packets by responder.	Severe destination loss indicates victim link congestion, server socket backlog overflow, or receiver buffer starvation under attack.

4. Inter-Packet Timing & Jitter Dynamics (4 Features)

Feature	Type	Metric Definition & Units	Physical Network Semantics & Security Analysis
`sinpkt`	`float64`	**Source Inter-Packet Time:** Mean interval (milliseconds) between consecutive source packets.	Automated attack scripts fire packets at deterministic, sub-millisecond intervals ($sinpkt \to 0$), contrasting with human interactive pauses.
`dinpkt`	`float64`	**Destination Inter-Packet Time:** Mean interval (milliseconds) between consecutive responder packets.	Measures server response consistency. Large `dinpkt` values indicate delayed processing, thread blocking, or server resource exhaustion.
`sjit`	`float64`	**Source Jitter:** Variance / standard deviation in packet arrival intervals (ms) from source.	Low jitter signifies synthetic, bot-driven packet generation; high jitter reflects route oscillations or randomized evasion tactics.
`djit`	`float64`	**Destination Jitter:** Variance in packet arrival intervals (ms) from responder.	Reveals responder queuing volatility, server load fluctuations, and TCP congestion window throttling under adversarial conditions.

5. TCP Handshake & Flow Control Dynamics (7 Features)

Feature	Type	Metric Definition & Units	Physical Network Semantics & Security Analysis
`swin`	`int64`	**Source TCP Window Size:** Advertised receive window buffer size (bytes) from initiator.	Zero window advertisements (`swin = 0`) signal client buffer collapse or socket stalls. Non-TCP flows record zero.
`dwin`	`int64`	**Destination TCP Window Size:** Advertised receive window buffer size (bytes) from responder.	A target under heavy DoS will advertise zero window sizes as its internal kernel socket queues become saturated.
`stcpb`	`int64`	**Source TCP Base Sequence (ISN):** 32-bit TCP base sequence number chosen by client.	Key fingerprint for OS identification. Custom attack scripts often generate pseudo-random or predictable initial sequence numbers.
`dtcpb`	`int64`	**Destination TCP Base Sequence:** 32-bit TCP base sequence number chosen by responder.	Zero in half-open SYN connections where the server never replied with a `SYN–ACK`.
`tcprtt`	`float64`	**TCP Round-Trip Time:** 3-way handshake setup duration in seconds: $tcprtt = synack + ackdat$.	Measures connection latency. Large `tcprtt` indicates network congestion or server processing delays under attack.
`synack`	`float64`	**SYN to SYN-ACK Latency:** Time gap (seconds) between client's `SYN` and server's `SYN–ACK`.	Direct diagnostic of server health. High `synack` denotes that the victim's SYN backlog queue is filling up under flood conditions.
`ackdat`	`float64`	**SYN-ACK to Final ACK Latency:** Time gap (seconds) between server's `SYN–ACK` and client's final `ACK`.	Attack scripts using raw sockets often skip the final ACK to keep sockets open (half-open scan), leading to null or anomalous latencies.

6. Application-Layer Content Indicators (5 Features)

Feature	Type	Metric Definition & Units	Physical Network Semantics & Security Analysis
`trans_depth`	`int64`	**HTTP Pipelining Depth:** Sequential HTTP request count over a single persistent TCP connection.	High transaction depth suggests HTTP pipeline exploitation, web crawling, or rapid-fire brute forcing over an established keep-alive channel.
`response_body_len`	`int64`	**HTTP Response Body Length:** Content length (bytes) of HTTP payload returned by web daemon.	Abnormally large response bodies indicate file leakage or exfiltration; zero-length bodies with 200 OK often indicate command execution.
`ct_flw_http_mthd`	`int64`	**HTTP Method Count:** Number of flows with standard HTTP request verbs (e.g., `GET`, `POST`).	Flags web application traffic density. Useful for isolating web exploit probes from non-web protocol tunneling.
`is_ftp_login`	`int64`	**FTP Authentication Flag:** Binary flag (1 if FTP session authenticated with credentials, else 0).	Zero values during extensive FTP traffic point to automated credential brute-force attacks or anonymous login attempts.
`ct_ftp_cmd`	`int64`	**FTP Command Count:** Number of commands issued in session (e.g., `USER`, `PASS`, `RETR`, `STOR`).	High command counts indicate automated directory transversal, bulk downloading, or command injection via FTP.

7. Windowed Connection Aggregates & Behavioral Anomalies (3 Features)

Feature	Type	Metric Definition & Units	Physical Network Semantics & Security Analysis
`ct_src_dport_ltm`	`int64`	**Source-to-Dest-Port Window:** Connections sharing SrcIP and DstPort in 100-flow window.	Detects distributed Denial of Service (DDoS) campaigns targeting a specific service port (e.g., port 80/443 or DNS port 53).
`ct_dst_sport_ltm`	`int64`	**Dest-to-Source-Port Window:** Connections sharing DstIP and SrcPort in 100-flow window.	Primary signature of port sweep scanning. An adversary scanning thousands of destination IPs using a fixed source port creates massive spikes.
`is_sm_ips_ports`	`int64`	**Same IP & Port Flag:** Binary flag (1 if $\text{SrcIP} == \text{DstIP}$ and $\text{SrcPort} == \text{DstPort}$).	Direct detection of classic **LAND attacks** (Local Area Network Denial), where spoofed packets cause a victim host to loop endlessly into itself.

8. Ground Truth Target Labels (2 Features)

Feature	Type	Metric Definition & Values	Physical Network Semantics & Security Analysis
`label`	`int64`	**Binary Target:** `0` = Benign (37,000) • `1` = Malicious (45,332).	Primary objective target for binary intrusion detection classifiers and rapid threat filtering.
`attack_cat`	`string`	**Multiclass Target:** 10 classes (Normal, Generic, Exploits, DoS, etc.).	Target for fine-grained threat attribution, forensic root-cause analysis, and automated security playbooks.

SEC 3 The 48 Engineered Features in the X-NIDS Architecture

Standard baseline features alone struggle with minority attack recall (e.g., Worms, Shellcode, Backdoor) due to extreme class imbalance and non-linear traffic patterns. The X-NIDS engine introduces **48 domain network features** engineered directly from raw network flows (implemented in `engine/train_and_save_model.py`):

DOMAIN 1 Volume Aggregation & Differences (5 Features)

`tot_bytes = sbytes + dbytes`
`tot_pkts = spkts + dpkts`
`tot_loss = sloss + dloss`
`byte_diff = |sbytes - dbytes|`
`pkt_diff = |spkts - dpkts|`

Captures total flow mass and unidirectional payload skewness.

DOMAIN 2 Asymmetric Flow & Loss Ratios (7 Features)

`bytes_ratio = (sbytes + 1) / (dbytes + 1)`
`pkts_ratio = (spkts + 1) / (dpkts + 1)`
`sbytes_per_pkt = sbytes / (spkts + 1)`
`dbytes_per_pkt = dbytes / (dpkts + 1)`
`loss_ratio = tot_loss / (tot_pkts + 1)`
`sloss_rate = sloss / (spkts + 1)` · `dloss_rate = dloss / (dpkts + 1)`

DOMAIN 3 Rate & Dynamic Throughput Ratios (5 Features)

`byte_rate = tot_bytes / (dur + 1e-5)`
`pkt_rate = tot_pkts / (dur + 1e-5)`
`sload_to_rate = sload / (rate + 1e-5)`
`dload_to_rate = dload / (rate + 1e-5)`
`load_ratio = (sload + 1) / (dload + 1)`

Isolates burst transmission dynamics from persistent traffic.

DOMAIN 4 TCP Handshake Latencies & Jitter Dynamics (10 Features)

`tcprtt_to_dur = tcprtt / (dur + 1e-5)`
`synack_to_ackdat = (synack + 1e-5) / (ackdat + 1e-5)`
`tcprtt_synack_ratio = synack / (tcprtt + 1e-5)`
`syn_ack_diff = |synack - ackdat|`
`jit_diff = |sjit - djit|` · `jit_ratio = (sjit + 1) / (djit + 1)` · `jit_sum = sjit + djit`
`dinpkt_to_sinpkt` · `dinpkt_dload_ratio` · `mean_size_ratio`

Remaining Engineered Feature Domains (Domain 5 & 6)

DOMAIN 5 **Connection & Window Metrics (3 Features)**

`ct_ratio = (ct_src_dport_ltm + 1) / (ct_dst_sport_ltm + 1)`
`swin_dwin_ratio = (swin + 1) / (dwin + 1)`
`stcpb_dtcpb_diff = |stcpb – dtcpb|`

Highlights port scan asymmetry and window exhaustion.

DOMAIN 6 **Binary Protocol Behavior Flags (4 Features)**

`has_tcp_handshake = 1 if tcprtt > 0 else 0`
`is_slow_handshake = 1 if tcprtt > 0.05 else 0`
`zero_dbytes = 1 if dbytes == 0 else 0`
`zero_dur = 1 if dur == 0 else 0`

Encodes protocol boundary states as clean linear indicators.

Log-Normal Variance Stabilizers (11 Features) & Categorical Frequency Maps (3 Features)

Heavy-tailed network distributions produce extreme gradient variance in decision trees. We apply **log-normal transforms** ($\log(1 + x)$) across 11 high-skew dimensions:

`log_sbytes`, `log_dbytes`, `log_tot_bytes`, `log_sload`, `log_dload`, `log_rate`, `log_dur`, `log_sinpkt`, `log_dinpkt`, `log_sjit`, `log_djit`

High-cardinality nominal variables (`proto`, `service`, `state`) are transformed into smooth continuous probability densities via **Frequency Encoding** with a minimum frequency cutoff ($min_freq = 50$).

SEC 4 Machine Learning Architecture & SHAP Explainability Engine

MODELING **Soft-Voting Ensemble & Class Weighting**

The detection engine fuses **XGBoost** (depth=7, lr=0.04) and **LightGBM** (leaves=127, min_child=20) through a calibrated soft-voting ensemble. To eliminate minority starvation (e.g., Worms with 44 samples), we implement **Power Class Weighting**:

$$w_c = (N / (C \cdot N_c))^{0.40}$$

Achieves balanced decision boundaries without synthetic SMOTE artifacts.

EXPLAINABILITY **SHAP TreeExplainer & Local Attributions**

For every flagged intrusion flow, `shap.TreeExplainer` computes exact game-theoretic Shapley values:

$$f(x) = \varphi_0 + \sum_{i=1}^{M} \varphi_i(x)$$

Identifies the **Top-3 Driver Features** (e.g., elevated `sload_to_rate`, zero `dbytes`, extreme `ct_dst_sport_ltm`), rendering human-readable alert cards that eliminate black-box alert fatigue for SOC analysts.

End-to-End Pipeline Performance Matrix

Pipeline Stage	Input Artifact	Output Artifact	Runtime Latency	Operational Purpose
Simulation	Network Topology	`simulated_traffic.pcap`	~1.2 seconds	Synthesizes bidirectional Ethernet frames with realistic attack injections.
Flow Extraction	`simulated_traffic.pcap`	36 NetFlow Records	~2.4 seconds	Reassembles TCP/UDP conversations into bidirectional 5-tuple statistical flows.
Inference Engine	36 NetFlow Records	Predictions + Confidences	~0.8 ms / flow	Applies 48 feature transformations and ensemble soft-voting classification.
SHAP Attribution	Flagged Attack Flow	`shap_alerts/*.png`	~3.7 seconds	Computes Shapley values and renders visual bar summary plots for SOC analysts.

Repository: github.com/axant4323/IDS-with-SHAP • **Status:** Verified & Certified **Lead Author:** Akshant.A.A • **Engine:** 100% Pure Python